

项目复盘

1 自我介绍

面试官您好，我是黄思铭，来自北京航空航天大学数学专业，方向是数学与应用数学。我的研究方向是强化学习，同时对推荐系统也有深入研究。技术方面，我熟练掌握Python和PyTorch，也熟悉MATLAB和C语言。我的数学基础比较扎实，在本研阶段，数学分析、高等代数、数理统计和深度学习等理论课程都能获得95左右甚至更高的分数。接下来我将简单介绍一下简历里面提到的三个项目：

第一个项目是“基于元学习的推荐系统冷启动ID生成框架”。推荐系统中新上线的物品由于交互数据不足，导致ID embedding训练不好，传统做法是随机初始化ID embedding或者使用默认值，推荐效果差。我的项目设计了一个冷启动框架，用生成器根据物品的属性特征生成高质量的初始ID embedding。

项目的核心创新是用元学习MAML的方式训练生成器。我把学习每个item的ID embedding当做一个任务，通过学习多个老item的任务，生成器掌握了为item生成好的embedding的能力。

此外还有两个辅助优化：一是，通过随机mask特征槽进行数据增强作为正样本，利用对比学习优化增强生成器的泛化能力。二是用交互用户的平均embedding进行去噪，减轻噪声影响。

在MovieLens-1M数据集上，我的方法将新item的GAUC从提高了1个百分点。

第二个项目是：基于CVaR正则的风险敏感多智能体训练算法。针对多智能体强化学习中的不确定性导致高风险行为的问题，我设计了基于 CVaR 正则 的训练算法，用集成Critic网络构建价值分布，引入条件风险价值惩罚高风险状态。算法在MPE各种环境中碰撞次数下降10%。

第三个项目是卫星对抗环境设计，我设计并实现了一个基于JAX的卫星对抗仿真环境，模拟地球同步轨道上的多卫星对抗场景，环境遵循OpenAI Gym标准接口，可无缝集成主流强化学习算法。

以上就是我的自我介绍全部内容，谢谢！

2 项目介绍：基于元学习的冷启动推荐系统

对于新上线的物品，也就是冷启动item，它的数据不足，导致它的ID embedding训练得很差，进而导致推荐模型推不准。因为ID特征是每个item独有的，所以ID特征只能吃到自己的数据，在自己数据不足的情况下，就更新不好。但其他特征比如类别等于运动，这个特征可以吃到所有运动类item的数据，所以肯定更新得很好。

所以解决冷启动问题的本质，就是解决冷启动item的ID embedding训练不好的问题。传统做法是给新item随机初始化一个ID embedding，但这个embedding质量很差。我的项目就是设计一个冷启动框架，用生成器为新item生成一个良好的初始ID embedding，来替换随机初始化的较差的embedding，以此提高推荐效果。

具体来说，我的项目有三个核心优化点。第一个优化点是构建ID embedding生成器。生成器的输入是item的属性特征比如电影的上映年份标题类型等，输出是一个质量不错的ID embedding。这里的关键是，我用元学习MAML的方式来训练这个生成器。

什么是元学习呢？传统机器学习是一条条样本地训练，而元学习是以任务为单位训练，通过学习多个不同的任务，模型掌握了一种“学习的能力”，当新任务来临时，只需要少量数据就能快速适应。”。我把学习每个item的ID embedding当做一个任务，用多个老item的数据构造多个task，通过对不同task的学习，生成器就学会了如何在少样本情况下，为item生成好的ID embedding这个能力。

然而，在推荐系统中，毫无交互的物品连少量数据都没有，所以传统的元学习是不够的。传统MAML是怎么做的呢？它学习一个好的模型参数初始化，当新任务来临时，用少量样本对模型参数做梯度下降更新，然后在更新后的参数上计算loss，用这个loss反向传播去优化初始化参数。但这有个问题，推荐系统中很多新item是完全冷启动，连少量样本都没有，根本没机会做更新。

所以我做了改进。我不仅用更新后的loss，还同时用更新前的loss，把这两个loss加起来，一起反向传播去更新生成器的参数。并且，我不是更新整个冷启动生成器的参数，而是只更新ID embedding向量。具体训练流程是这样的：先用生成器为训练集的item生成初始ID embedding，在第一个batch数据上计算完全冷启动的loss，这时候还没有更新。然后用这个loss对ID embedding做一步梯度下降，得到更新后的ID embedding，再在第二个batch数据上计算预热阶段的loss。这样生成器在训练时就同时考虑了两个阶段，既能应对0样本的完全冷启动，也能应对有少量数据的预热阶段。当新item来临时，直接用生成器为它生成ID embedding，这个embedding质量就比随机初始化好很多。如果新item积累了少量数据，还可以在这些数据上对ID embedding做一步梯度下降，让它快速适应到新item的真实特征，进一步提升效果。

还有两个优化点相当于是辅助技巧。第一个是基于对比学习的特征空间优化。我把item的属性特征分成两个视图作为数据增强的手段，虽然这两个视图看到的信息不同，但它们描述的是同一部电影，所以应该生成相似的embedding，这是正样本对。同时，我把batch内其他item的视图作为负样本，强制当前item的embedding和其他item的embedding保持距离。通过InfoNCE Loss，拉近正样本对，推远负样本对，这样可以优化embedding空间的几何结构，让相似的item聚在一起，不相似的item分开，增强生成器的泛化能力。

第二个辅助技巧是嵌入去噪。对于冷启动item来说，交互用户数量本来就有限，噪声用户的影响会非常大。我的做法是用冷启动item的最近20个交互用户的平均id embedding来进行去噪。将其拼接t到生成器生成的id embedding上，再过一层MLP网络，可以减轻离群点的影响，从而起到降噪的作用。

训练流程分两个阶段。在讲训练流程之前，要先讲数据集进行处理，以MovieLens-1M电影评分数据集为例，我把交互数大于512的电影当做老item占90%，小于等于512的当做新item占10%作为冷启动

的测试集。而老item中再取90%的数据作为**基础DNN模型的训练集**，剩下10%分为两部分作为冷启动训练集。

第一阶段是**预训练推荐模型**，用老item的大量数据训练一个基础的DNN模型，这个阶段就是正常的推荐模型训练。第二阶段是**元学习训练生成器**，冻结第一阶段训练好的DNN参数，只训练生成器的参数。这里用的是MAML的训练方式，每个老item取两个batch的数据，第一个batch模拟**完全冷启动阶段**，用生成器生成的embedding进行预测，计算**冷启动损失**。然后让这个embedding在冷启动损失上梯度下降一步，模拟数据积累后embedding得到更新。再用第二个batch计算**预热损失**，评估更新后的embedding效果。最后用这两个损失加上**对比学习损失**，一起反向传播更新生成器的参数。

预测的时候，当新item来临，直接用生成器根据它的属性特征生成ID embedding，然后用这个embedding参与推荐。如果新item已经有了少量数据，还可以在这些数据上做一步梯度下降，让embedding快速适应到新item的真实特征，进一步提升效果。

实验结果显示，不用冷启动框架的话，新item的**GAUC是0.84**。加入我的冷启动框架后，**GAUC提升到0.85**，提高了1个百分点。

3 AUC与GAUC指标详解

3.1 什么是AUC

AUC(Area Under the ROC Curve)是**ROC曲线下的面积**，是评估二分类模型最常用的指标之一。

ROC曲线:

- 横轴: $FPR(False\ Positive\ Rate) = \frac{FP}{FP+TN}$, 假正例率
- 纵轴: $TPR(True\ Positive\ Rate) = \frac{TP}{TP+FN}$, 真正例率(也叫召回率)

AUC的物理意义:

AUC等于随机抽取一个正样本和一个负样本,正样本的预测分数大于负样本的概率。

数学表达:

$$AUC = P(score_{pos} > score_{neg})$$

AUC的优点:

- **阈值无关**: 不需要选择分类阈值,评估模型的整体排序能力
- **类别不平衡鲁棒**: 相比准确率,AUC对正负样本比例不敏感
- **概率解释**: 有明确的概率含义,易于理解

AUC的计算方法:

除了画ROC曲线计算面积,还有更高效的**基于排序的计算方法**:

1. 将所有样本按预测分数升序排序
2. 计算所有正样本的秩次和(rank sum)
3. 使用公式: $AUC = \frac{rank_sum - \frac{n_{pos}(n_{pos}+1)}{2}}{n_{pos} \times n_{neg}}$

其中 n_{pos} 是正样本数, n_{neg} 是负样本数。这个公式的本质是统计有多少个(正样本,负样本)对满足正样本分数更高。

3.2 为什么推荐系统要用GAUC

GAUC(Group AUC)是推荐系统中更常用的指标,它是按用户分组计算AUC后加权平均。

为什么不能直接用AUC?

在推荐系统中,直接用全局AUC有严重问题:

问题1: 用户间样本不可比

推荐系统的本质是为每个用户排序候选物品。不同用户看到的物品集合不同,他们的点击行为也不可比。比如:用户A特别活跃,看到什么都容易点;用户B很冷淡,基本不点

- 用户A的一个未点击样本: 0.8
- 用户B的一个点击样本: 0.6

全局AUC在此时会产生误判。

问题2: 活跃用户主导指标

假设用户A有1000条交互记录,用户B只有10条。全局AUC中,用户A贡献了 $1000 \times 1000 = 100$ 万个正负样本对,用户B只贡献了 $10 \times 10 = 100$ 个。这导致活跃用户完全主导了指标,模型只要把活跃用户服务好,AUC就很高,但可能对长尾用户效果很差。

问题3: 忽略个性化

推荐系统的核心是个性化,每个用户的偏好不同。全局AUC把所有用户混在一起,无法反映模型是否真正学会了个性化推荐。

GAUC的解决方案:

GAUC的计算流程:

1. 按用户分组: 将数据按user_id分组
2. 计算每组AUC: 对每个用户单独计算AUC,只比较该用户看到的物品
3. 加权平均: 用每个用户的正负样本数乘积作为权重,计算加权平均AUC

数学表达:

$$GAUC = \frac{\sum_{u \in Users} AUC_u \times (n_{pos}^u \times n_{neg}^u)}{\sum_{u \in Users} (n_{pos}^u \times n_{neg}^u)}$$

GAUC的优点:

- **符合推荐场景:** 只比较同一用户看到的物品,符合推荐系统的实际使用场景
- **公平性:** 每个用户的权重与其样本数成正比,避免活跃用户主导
- **个性化评估:** 能够反映模型是否真正学会了为每个用户个性化排序
- **长尾友好:** 对长尾用户和头部用户一视同仁

为什么冷启动项目用GAUC:

在我的冷启动项目中,使用GAUC有特殊意义:

- **评估公平性:** 冷启动item的交互数据少,如果用全局AUC,它们的贡献会被热门item淹没
- **用户视角:** 冷启动的目标是让每个用户都能发现新物品,GAUC能反映每个用户对新item的接受度

- **避免偏差**: 防止模型只优化热门用户对新item的推荐,而忽略长尾用户

GAUC的局限性:

- **计算成本**: 需要按用户分组计算,比全局AUC慢
- **单正样本用户**: 全正全负无法计算AUC,需要过滤
- **权重选择**: 用正负样本数乘积作为权重是一种选择,也可以用其他权重(如按曝光数或样本数加权)

总结: AUC评估模型的全局排序能力,GAUC评估模型的个性化排序能力。在推荐系统中,GAUC更能反映模型的真实效果,因为它符合“为每个用户个性化排序”的业务场景。我的冷启动项目使用GAUC,能够公平地评估新item在不同用户群体中的推荐效果,避免被活跃用户或热门item主导。

4 为什么强化学习课题组要转投搜广推？

我在强化学习课题组期间,在学习过程中,我发现**推荐系统天然就是一个序列决策问题**:“状态”是系统当前能看到的用户上下文:用户画像、历史交互序列等信息。“动作”就是系统这一步“决定推荐什么、怎么推荐”。奖励就是你优化的业务目标:点击率、停留时长、观看时长、点赞、关注、分享等等。这让我意识到**强化学习和推荐系统有天然的契合点**,激发了我对搜广推领域的兴趣。

强化学习的思维方式对推荐系统很有帮助。首先是**探索利用平衡**,正好对应推荐系统中探索新item和推荐老item。其次是**长期价值优化**,不只看当前点击率,还要考虑用户留存。第三是**序列建模能力**,用户行为是序列的,需要捕捉时序依赖。我做的这个冷启动项目,虽然用的是元学习,但本质上也是在解决如何快速适应新任务的问题,和强化学习中的**few-shot** 思想是相通的。

近年来,我特别关注到**生成式推荐的发展趋势**。传统推荐范式是召回粗排精排重排的多阶段链路,每个阶段独立优化,存在信息损失,依赖大量人工特征工程。而生成式推荐可以端到端生成推荐结果,减少由于多阶段流水线拆分而额外引入的性能损失,大模型的涌现能力能更好理解用户意图,还可以生成个性化的内容描述,提升用户体验。

我注意到工业界已经在大规模应用RL到推荐系统,比如YouTube用DQN做视频推荐,阿里用虚拟淘宝环境训练推荐策略,字节用Contextual Bandit做冷启动,小红书快手在探索生成式推荐。这让我看到了**强化学习在搜广推领域的巨大应用价值**,也坚定了我转向这个方向的决心。

从就业角度,搜广推是互联网公司的核心业务,岗位需求大,技术栈成熟。而且我的强化学习背景是加分项,能带来差异化竞争力。特别是在生成式推荐这个新兴方向,强化学习的探索能力有用武之地。

5 对比学习的正样本构造与语义一致性

我理解面试官的核心疑问是:我把item的属性特征分成两个子集view1和view2,用这两个不完整的特征子集分别生成embedding,然后强制这两个embedding相似,但两个子集信息不同,生成的embedding凭什么要相似,这不是破坏了语义吗?

首先我解释一下**对比学习的本质**。对比学习的核心就是**数据增强**,对同一个样本做不同的变换,得到多个视图,这些视图表面形式不同,但**本质语义相同**。然后强制模型学习,这些不同视图应该映射到相似的表示,从而让模型忽略表面差异,抓住本质特征。比如CV领域,同一张猫的图片可以旋转裁剪变

色，表面像素不同，但都是猫这个语义。通过对比学习，模型学会不管角度颜色如何变，都能识别出是猫。

我的方法本质上和标准对比学习的数据增强是一样的，只是**增强方式不同**。CV可以用旋转裁剪，NLP可以用回译同义词替换，但推荐系统的特征是离散的，不能像图像那样连续变换。我采用的数据增强方式是**随机mask掉一部分特征槽**，这样同一新item的两个视图看到的信息不同，但它们描述的是**同一部电影**，这就是数据增强。

但这里有个问题，如果直接把某些特征mask成0，可能会完全改变电影的属性表示，相当于变成了另一部电影，破坏了语义。为了解决这个问题，我加入了一个**SENet风格的注意力机制W-SENET**。它的作用是，当某些特征被mask成0之后，模型可以**自适应地调整剩余特征的权重**，给重要的特征更高的权重，从而补偿被mask掉的信息。这样即使mask掉了一部分特征，生成的embedding也不会完全偏离原来的语义，而是保持在合理的范围内。

通过对比学习，我强制生成器学习，不管从哪个角度看，也就是不管用view1还是view2，都应该生成相似的ID embedding。这和CV中强制不同角度的图片生成相似的表示是一个道理。这样做的好处是什么呢？生成器学会了从**不完整信息中提取本质特征**。当新item来临时，即使某些特征缺失，生成器也能生成合理的embedding，这就提升了鲁棒性和泛化能力。

总结一下，我的方法和标准对比学习的数据增强本质是一样的，都是对同一个样本做不同的变换，然后强制不同变换生成相似的表示。只是CV用旋转裁剪，我用特征子集划分，但**核心思想是一致的**。这不是破坏语义，而是让模型学习**语义的不变性**，从而提升泛化能力。

5.1 为什么要做对比学习

我使用对比学习主要是为了优化embedding空间的结构，让生成器生成的ID embedding更有区分度。

对比学习的本质是通过同源相近、异源相远来优化表示空间。同源相近是指同一个item的不同表示应该靠近，异源相远是指不同item的表示应该分开。这样可以让embedding空间更有序，相似的item聚在一起，不相似的item分开。以防止出现相似的电影生成的embedding距离很远，不相似的电影反而很近，embedding空间混乱的情况。

6 冷启动是user-item，为什么你做item-item？

冷启动问题确实是预测**user-item交互**，但解决方案可以从不同角度切入。传统的**user-item建模思路**是协同过滤，输入user id和item id，输出预测评分或点击概率，但问题是**新item没有user交互历史**，协同过滤失效。而**item-item建模**的思路是内容增强，核心是找相似老item迁移它们的知识，输出新item的**良好ID embedding**，然后用这个embedding参与user-item预测。

我的方案是**item-item是手段，user-item是目标**。完整流程分两个阶段。阶段一是**item-item相似度**，也就是生成器训练，新item找到K个相似老item，生成新item的ID embedding。阶段二是**user-item预测**，也就是推荐模型，把user特征、新item的生成embedding、其他特征输入DNN，预测点击率。

为什么强调item-item呢？第一，**冷启动的本质是item端问题**。新item没数据，user是有数据的，问题出在item的ID embedding训练不足，所以要从item端解决。第二，**item-item是手段，user-item是目标**。通过item相似性生成embedding是手段，最终还是预测user对item的兴趣是目标。第三，元学习

的任务定义，每个task是学习一个item的embedding，不是学习一个user的偏好，因为item是冷启动的，user不是。

总结一下，不是只有item-item，而是**item-item相似生成item embedding**，然后**user-item**预测，用item-item解决user-item问题。如果是user冷启动，思路会反过来，**user-user**建模找相似老用户生成新用户的embedding，然后用这个user embedding参与user-item预测。但我的项目聚焦**item冷启动**，所以是item-item建模。

7 为什么要用第一段Loss，既然完全冷启动，那应该没有用户也就没有label？

第一段Loss用的不是真正0数据的冷启动item，而是用老item的历史数据来模拟冷启动场景。具体来说，我用生成器为老item生成一个embedding，替换掉它预训练好的embedding，然后在它的一部分历史交互数据上计算loss。这模拟的是：如果这个item是新的，只能用生成器生成embedding，效果如何。通过优化这个loss，生成器学会了即使没有预训练，也要生成高质量的初始embedding。这样当真正的新item来临时，生成器就能为它生成一个好的起点。

第二段Loss模拟的是预热阶段。在第二段Loss的基础上，我让生成的embedding在第一批数据上做一步梯度下降，得到更新后的embedding。然后在老item的另一部分历史交互数据上计算loss。这模拟的是：当新item积累了一些数据后，embedding经过更新，效果能提升多少。通过优化这个loss，生成器学会了不仅要生成好的初始embedding，还要生成容易适应的embedding，也就是说，这个初始embedding在少量数据上更新后，能快速达到好的效果。这样当真正的新item积累了少量数据时，就能快速适应到最优状态

8 在训练过程中，并行化方面遇到了什么困难？怎么解决的？

8.1 冷启动

在我的项目中，我使用了TFRecord格式存储数据。选择TFRecord主要有几个原因：第一，它是二进制格式，读取速度比CSV快。第二，存储效率高，文件体积更小，适合大规模数据的顺序流式读取。第三，TensorFlow生态支持好，有现成的读取库。第四、天然适合切分成多个文件做并行 I/O

不过因为数据规模不大，单进程已经不是瓶颈，单进程的简单性和稳定性更重要。当然，如果是工业界的大规模数据，肯定需要多进程甚至分布式加载。”

8.2 强化学习

强化学习的瓶颈通常在环境交互，所以主要在环境交互上进行了并行化，主要用了2个技术来解决。

第一个是 replay buffer 的线程锁。因为我做的是多智能体强化学习，多个 agent 会同时采样数据并写入共享的 replay buffer。如果不加锁，就会出现 race condition，比如两个线程同时读到相同的 current-size，然后写入同一个索引位置，导致数据被覆盖。我用线程锁把写入操作保护起来，确保同一时刻只有一个线程能修改 buffer，这样就避免了数据损坏的问题。

第二个是用 pathos 的 ProcessingPool 做多进程环境采样。强化学习的瓶颈通常在环境交互上，单进程跑环境太慢了。使用多进程环境采样就能同时跑多个环境实例，把采样速度提升了好几倍。

8.3 卫星对抗

选择 JAX 同样是因为这个项目的瓶颈在环境仿真而不是神经网络本身。第一，我们的环境里包含基于 CWH 方程的动力学积分（属于固定维度、低分支、可批量复制的数值动力学，刚好是 JAX 舒适区），项目对计算时间要求极高，DiffraX 是 JAX 生态里的可微分方程求解库，相比其余解方程的库，jax 是唯一满足时间要求的。第二，我们需要同时并行大量卫星对抗环境，JAX 的 vmap 很适合把单环境 step 自动提升为批环境计算，只要环境写成纯函数形式，就能比较自然地在 GPU 上并行执行。

9 在训练过程中，遇到最大的困难是什么？怎么解决的？

9.1 冷启动

项目中最大的挑战是损失函数的设计。最初我按照经典 MAML 只用第二段 loss 来训练，但发现当新 item 完全没有交互数据时，推荐效果根本没好转。分析后发现，经典 MAML 假设新任务至少有少量样本可以进行梯度更新，但推荐系统的冷启动更极端——新 item 可能在最初几小时内都是零数据，根本没法做梯度更新，只能直接用生成的 embedding 预测。

为了解决这个问题，我改进了损失函数，加入了第一段 loss。第一段 loss 约束生成器输出的初始 embedding 本身就要好用（应对零数据场景），第二段 loss 约束更新后的 embedding 要更好（应对少量数据场景）。这样生成器需要同时优化初始质量和快速适应能力两个目标。

9.2 强化学习

前期对不确定性量化的学习

9.3 卫星对抗

问题描述：我遇到的最大困难是奖励函数设计不好导致智能体学会了消极策略。具体表现是训练初期智能体不敢接近敌人，总是往战场边缘跑，甚至学会了在边界附近徘徊拖时间。这完全违背了我们希望它主动作战的初衷。

问题分析：我分析后发现主要有三个原因。第一是攻击敌人的奖励相比被敌人攻击的惩罚没有优势，智能体算了一笔账发现“进攻可能被打”和“进攻打中敌人”的收益差不多，那还不如不动。第二是没有范围惩罚，智能体可以无限远离战场中心，只要不出界就没事，所以它学会了躲在角落里。第三是缺少时间成本，智能体可以无限拖延，最后学会了拖到300步超时的策略。

方案1——探测奖励：我首先加了探测奖励来引导智能体主动接近敌人。每探测到一个敌方卫星就给一个较低的奖励，虽然不多，但足够让智能体在早期就能获得正反馈。这样它就不会一直躲在角落里，而是会主动靠近敌人去侦察。

方案2——时长惩罚：然后我加了时长惩罚，每步扣0.1分，并且如果拖到一个回合结束将给予一个很大的惩罚。这个设计直接打破了智能体的拖延策略，因为拖得越久扣得越多，它必须速战速决。这个惩罚对红方所有存活的战斗卫星都生效。

方案3——奖励分层：任务设定是保护一个中心卫星，根据敌方卫星到需要保护的卫星距离进行分层惩罚。这样智能体就不会等敌人冲到脸上才反应，而是学会了前置防御，主动在外围拦截敌人。

9.3.1 怎么判断奖励设计的好坏？

我主要通过对抗不同蓝方策略来验证奖励设计的好坏。我的环境里蓝方有多种预定义的策略，每次训练时会随机选择一个策略作为对手。（例如：随机游走、优先进攻、优先逃跑、诱敌深入、优先攻击保护卫星等基于规则的算法）如果奖励设计得好，红方智能体应该能够适应各种蓝方策略，而不是只会打某一种。

具体验证方法：训练时，每个episode开始时先从蓝方策略库里面挑选一个策略。这样红方在训练过程中会遇到各种不同风格的对手，比如激进型的、防守型的、游击型的等等。

判断标准1——泛化能力：如果奖励设计得好，红方训练出来后对所有蓝方策略的胜率应该比较均衡，不会出现“打得过A策略但打不过B策略”的情况。我会统计红方对每种蓝方策略的胜率，如果方差很大，说明奖励设计可能过拟合到某种特定对抗模式了。

判断标准2——策略多样性：好的奖励设计应该让智能体学会根据对手调整策略。比如遇到激进的蓝方就打防守反击，遇到保守的蓝方就主动进攻。我会看训练日志里的行为模式，如果智能体不管遇到什么对手都用同一套打法，说明奖励设计太单一了，没有鼓励它学习多样化的策略。

判断标准3——收敛速度：如果奖励设计得好，智能体应该能在合理的训练步数内收敛到一个稳定的策略。我会看训练曲线，如果胜率一直震荡不收敛，或者收敛得特别慢，说明奖励信号可能太稀疏或者相互冲突了。比如我加了探测奖励和时长惩罚后，收敛速度明显加快了。

实际案例：最初我只有击杀和获胜奖励时，红方只能打赢那些比较笨的蓝方策略，遇到会走位的蓝方就完全不行。再加入上述惩罚后，红方对各种蓝方策略的胜率都提升了，而且学会了根据对手调整打法。这说明新的奖励设计更好，因为它鼓励了更通用的能力，而不是针对某种特定对手的技巧。

10 冷启动在工业上一般如何处理？

工业里一般至少区分三类：新用户冷启动、新物品冷启动，以及新场景/新品类冷启动。它们的难点不一样，所以处理方式也不同。比如 Spotify 在做 audiobook 推荐时，面对的就不是普通的新 item，而是“整类新内容”带来的极端稀疏问题；Netflix 近年的推荐基础模型也明确把新 title 的 cold start 当成核心能力之一。

对新用户冷启动，工业上最常见的第一步不是强行个性化，而是先用热门/新鲜/规则兜底，再尽快收集少量偏好信号。这个阶段常用的信号包括注册信息、设备、地域、时间、入口、当前 session 行为、首屏点击、短期停留等；如果业务允许，还会加 onboarding 问题或兴趣选择。随后系统会用这些浅层特征去构造一个“可迁移的用户表示”，而不是等这个用户积累大量历史再开始推荐。Spotify 2024 的公开工作就是把多种用户特征压成通用表示，再迁移到下游推荐任务中使用。

对新物品冷启动，工业上最常见的是先不用 item ID 学它，而是先靠侧信息/内容信息给它一个初始表示。也就是把标题、类目、品牌、价格、标签、文本、图片、音频、多模态内容、商家属性等映射成 embedding 或语义特征，先让新物品“按内容相似性”进入召回和排序。Alipay 的 SIGIR 2022 应用论文就属于这一路：先建立从属性特征到冷启动特征空间的全局语义映射，再在线上补齐真实反馈。Amazon 的 MAREC 也明确是用 metadata 去增强冷启动推荐。

如果是新品类/新域冷启动，工业上很常见的办法是跨域迁移。也就是新域本身没数据，但平台别的域有大量行为，于是把老域的用户偏好迁移过来。Spotify 的 audiobook 推荐就是典型案例：它直接利用用户在 podcast 和 music 上的偏好来支撑 audiobook 推荐，在大规模线上实验里带来了明显增益。这个思路在工业里很常见，因为很多平台不是“完全没数据”，而是“新业务没数据，但老业务有数据”。

再往后就是探索流量。工业系统不会只靠静态特征，因为冷启动的本质还是“缺反馈”。所以通常会专门留一小部分流量做探索，让新用户、新物品尽快获得真实曝光和交互。这里常见的方法是 contextual bandit、受控随机、progressive feedback，或者在重排阶段加新颖性/探索项。Spotify 2025 的公开工作就展示了在工业推荐里利用 progressive feedback 和 bandit 式思路优化探索，可以带来线上收益。

真正上线时，工业系统一般不是“冷启动模型”和“成熟模型”二选一，而是分阶段混合。刚上线时，分数更多依赖热门度、内容特征、跨域相似用户、规则白名单；拿到少量交互后，逐渐提高协同过滤、ID embedding、序列模型、用户长期兴趣模型的权重；等样本足够后，再完全转入正常的召回-粗排-精排链路。Netflix 近年的公开内容也强调，新内容需要在没有交互前先具备可推荐能力，而后再随着反馈积累不断校准